



A conflict-aware active BLE state machine learning and state coverage-guided gray-box fuzz testing method

Long Yin¹ · Jian Xu¹ · Heqiu Chai¹ · Zhongsheng Wang¹ · Chunyu Liu¹

Published online: 21 May 2025

© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

Abstract

Bluetooth Low Energy (BLE) is extensively utilized in IoT and industrial control, yet its rapid advancement has exposed critical security flaws in protocol stack implementations, endangering user privacy and safety. Existing detection techniques, including state machine learning, differential testing, and fuzzing, struggle to accurately model complex BLE device behavior due to limited interaction data. Additionally, conventional fuzz testing achieves only shallow protocol state coverage and fails to handle BLE response uncertainties in dynamic environments. To address these issues, we propose a conflict-aware active learning approach based on the Minimally Adequate Teachers (MAT) framework. This method constructs an initial BLE protocol state machine using a cache tree, generates query sequences via a discrimination tree, and integrates an observation tree with a probabilistic model for conflict resolution. Additionally, we introduce a state coverage-driven fuzz testing strategy that utilizes the learned state machine and protocol semantics for automated test case generation. An evolutionary algorithm further optimizes test case mutation and execution order. Experimental results demonstrate that our method identifies more representative vulnerabilities in real BLE devices than three existing BLE fuzzers, providing an efficient and scalable solution for automated security analysis in BLE devices.

Keywords Bluetooth low energy · Grey-box fuzzing · Active automata learning · State coverage

1 Introduction

Bluetooth Low Energy (BLE) has become essential for IoT advancements, valued for its low power consumption, cost-effectiveness, and high convenience. It is integrated into billions of devices across sectors like healthcare, smart homes, and industrial automation. However, its widespread adoption has also exposed critical security risks, drawing significant interest from researchers and industry professionals. Studies have identified severe vulnerabilities in BLE chip protocol stacks. For example, Garbelini et al. [7] uncovered

weaknesses in multiple BLE chip implementations that let attackers bypass security protocols and interfere with device functionality, impacting over 480 products.

Recent studies discovered fundamental flaws in the implemented BLE protocol stacks, where inadequate security considerations during development have led to deviations from established design principles. Despite the security level upgrades in BLE 4.0 and the refinements in 4.2 and 5.0, the practical vulnerabilities remain prevalent. Many chip manufacturers often misinterpret or overlook standard specifications. This leads to discrepancies in how the protocol stack is executed. Hardware limitations, including restricted computational resources and constrained input mechanisms, often hinder devices from fully implementing all security features.

These vulnerabilities need efficient security testing. Fuzz testing uses automation to generate and input test cases and monitor responses. It is effective at identifying flaws and is well-documented [1, 5, 24]. As BLE continues to expand within the IoT landscape, the development of targeted and efficient fuzz testing methods for BLE devices is increasingly critical.

✉ Jian Xu
xuj@mail.neu.edu.cn

Long Yin
2110499@stu.neu.edu.cn

Heqiu Chai
2101229@stu.neu.edu.cn

Zhongsheng Wang
1141094862@qq.com

Chunyu Liu
839261192@qq.com

¹ Software College, Northeastern University, Shenyang 110169, China

The primary contributions of our research are as follows:

1. We present a conflict-aware active learning approach within the MAT framework, integrating cache, distinction, and observation trees with probabilistic models. This approach improves BLE state machine modeling by discovering 30% more states, cutting query overhead by 25%, accelerating convergence by 50%, and dynamically resolving state transition conflicts.
2. We propose a gray-box fuzz testing strategy that enhances protocol state coverage by combining state-guided test case generation with evolutionary scheduling. This method increases high-quality test cases by 30% and reduces vulnerability detection time by nearly 25%.
3. Our approach encodes mutation strategies for BLE packet fields and optimizes test execution using a weighted fitness function. Leveraging evolutionary algorithms, it improves test case generation and BLE fuzzing efficiency, enabling deeper protocol analysis under noisy conditions.

2 Related works

As a growing communication technology in the IoT ecosystem, Bluetooth Low Energy (BLE) security has garnered significant attention. Recent research has identified critical vulnerabilities in BLE protocol stacks and implementations using techniques such as manual inspection, static modeling, and fuzz testing.

Earlier studies primarily employed expert-driven methods, including code audits, hardware assessments, and firmware reverse engineering, to detect security flaws. For example, Seri and Vishnepolsky [17] found a memory corruption issue in BLE's broadcast packet parsing that could enable remote code execution. By studying different manufacturers' BLE firmware, they demonstrated how these vulnerabilities could be exploited to compromise enterprise WiFi networks.

Static modeling constructs formal representations of BLE protocol specifications to detect security weaknesses through model checking and formal analysis. Wu et al. [20] utilized ProVerif to systematically examine BLE security, revealing two major vulnerabilities in the reconnection process. They introduced the BLESa attack, which allows adversaries to impersonate paired server devices and inject forged data into clients, circumventing authentication. Later, Wu et al. [21] extended their model to cover key negotiation, data transmission, and additional protocols, including classic Bluetooth, BLE, and BLE Mesh, uncovering both known and newly identified security flaws.

Fuzz testing has proven effective in identifying critical vulnerabilities in software, but applying it to Bluetooth proto-

cols poses unique challenges. These include the need to inject timed packet sequences, proprietary protocol stack implementations that hinder gray-box or white-box testing [8, 22], and the inherent randomness and retransmission behaviors of wireless communication [6, 23]. Additionally, the isolation between the link and host layers complicates comprehensive testing [7].

To address these challenges, researchers have developed specialized tools and frameworks. Defensics [18] explored fuzz testing for classic Bluetooth but lacked optimization for efficiency and was limited in scope. Ruge et al. [16] introduced the Frankenstein framework for testing Bluetooth firmware systematically without physical devices, discovering three zero-day vulnerabilities in Broadcom and Cypress stacks. Garbelini et al. [7] developed a BLE controller firmware to abstract timing and retransmission constraints, enabling manipulation of link-layer packets for fuzz testing and revealing 11 new vulnerabilities across 480 devices. Lei et al. [12] proposed a fuzzing approach targeting the host implementation of BLE in Linux BlueZ by simulating a non-compliant software controller and building an HCI-based BLE model [11], [2], [13].

Park et al. [14] developed a state-guided fuzz testing tool targeting the L2CAP layer, identifying five zero-day vulnerabilities. Pferscher et al. [15] leveraged state machine learning to infer BLE device behavior, improving fuzzing coverage and effectiveness. Karim et al. [10] proposed BLEDiff, an automated differential testing framework based on active state machine learning, which identified 13 anomalous behaviors, including 10 exploitable vulnerabilities, across 25 commercial devices. Similarly, Shu et al. introduced IoTInfer, a black-box fuzzing framework guided by finite state machine inference, outperforming state-of-the-art tools like IoTFuzzer [3] and Snipuzz [4] in eliciting diverse responses from IoT devices. Verde et al. [19] assessed the risks of cyberattacks on BLE devices in healthcare and industry with an BLE-based asset management system prototype.

These aforementioned studies, while enhancing the understanding and mitigation of BLE security risks, also face several notable limitations. Many BLE fuzzing tools [8, 17, 21, 22, 24] rely heavily on the expertise and manual efforts of security researchers, making the process time-consuming and inefficient. Additionally, these tools are often tailored to specific chipsets, limiting their portability and applicability to devices from other manufacturers. Analyses based on predefined protocol models frequently idealize and abstract specifications, overlooking subtle flaws in real-world implementations and reducing their effectiveness in identifying specific implementation vulnerabilities. Furthermore, the unique challenges of wireless communication, including its unreliability and non-deterministic outcomes, are often inadequately addressed, particularly in methods that depend on active learning, thereby diminishing testing efficiency.

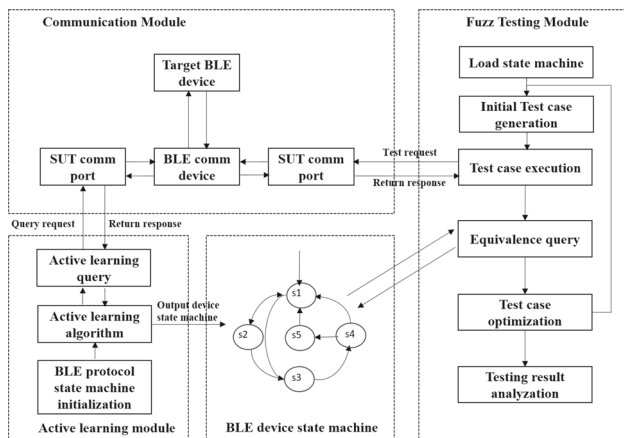


Fig. 1 The Proposed BLE Fuzz Testing System

3 Proposed BLE fuzz testing system

The proposed fuzz testing system comprises three key modules: active learning, fuzz testing, and communication. The communication module abstracts the system under test (SUT) and ensures stable BLE device interactions, allowing the active learning and fuzz testing modules to seamlessly transmit queries or test cases and process responses. The system workflow is illustrated in Figure 1.

The active learning module, employing a conflict-aware algorithm, constructs a Mealy state machine through interactions with the SUT. It consists of a scheduler, query manager, and active learner. The scheduler orchestrates communication between the query manager and active learner, overseeing the learning loop. The query manager generates and manages queries via a discrimination tree (guiding query formation), query generator, and cache tree (preventing redundant queries). The active learner updates the state machine, integrating a probability model, observation analyzer, and conflict resolver. The observation analyzer organizes new observations within an observation tree and detects inconsistencies, while the conflict resolver selects the most probable response or re-initiates queries when necessary.

The fuzz testing module handles test case generation, execution, optimization, and analysis. The model loader initializes the state machine for subsequent testing. Guided by state coverage feedback, the test case generator produces and refines cases using mutation algorithms. The test executor runs these cases, interacts with the target device, and records responses. The equivalence checker determines whether the device's actual state aligns with the expected model state, identifying anomalies. The test case optimizer enhances test cases through evolutionary algorithms involving selection, crossover, and mutation. Finally, the test analyzer assesses state coverage, detects defects, and generates a test report.

4 BLE device state machine generation

4.1 Conflict-aware active learning method

This section presents the workflow of conflict-aware active learning, as shown in Figure 2, which efficiently constructs the BLE device state machine to accurately capture its operational behavior. The process consists of three stages: initialization, active learning query, and conflict resolution. The cache tree, discrimination tree, observation tree, and probabilistic model collaboratively support its development.

(1) Initialization. In this phase, an initial model of the state machine is built using prior knowledge of the BLE protocol and stored within the cache tree. This model encapsulates common operational states and transitions under default settings, serving as a reference for subsequent learning steps. Meanwhile, a discrimination tree is created to assist in generating query sequences.

(2) Active Learning. In active learning, the method uses a discrimination tree to generate request sequences that best distinguish known states. The system checks the cache tree for reusable request-response paths. If found, the request is sent for verification; otherwise, a new sequence is executed. The obtained request-response pairs are stored in the observation tree, and the system verifies if responses match expectations. If not, a conflict is detected.

(3) Conflict Resolution. When conflicts occur between the cache and observation trees, the probabilistic model analyzes statistical data from the observation tree. It selects the most likely response exceeding the confidence threshold, then updates the cache tree to resolve the conflict.

The above-mentioned active learning cycle repeats until a max query counts limit or time threshold is reached, then outputs the finalized state machine of the tested BLE device.

4.2 Initialization of the BLE state machine

The Cache Tree stores the initial BLE state machine representation, query sequences, and transition paths. It retains finite prefixes to enable efficient reuse of past queries and transitions.

Definition 1 (Cache Tree) The rooted cache tree $T = (V, E)$, Each node $v \in V$ contains a protocol state $state(v)$, an input packet set $in(v) \subseteq I$, and an output packet set $out(v) \subseteq O$. An edge $e = (u, v) \in E$ signifies a transition from node u , in state $state(u)$, to node v , through sending $in(v)$ and receiving $out(v)$.

If there is a root node r that $in(r) = out(r) = \emptyset$ and $state(r)$ is uninitialized. For $\forall v \in V$, The input packet sequence $path(v)$ from r can be obtained by traversing the parent edges. The complete path $path(v)$ from r to v is expressed as follows in Equation 1:

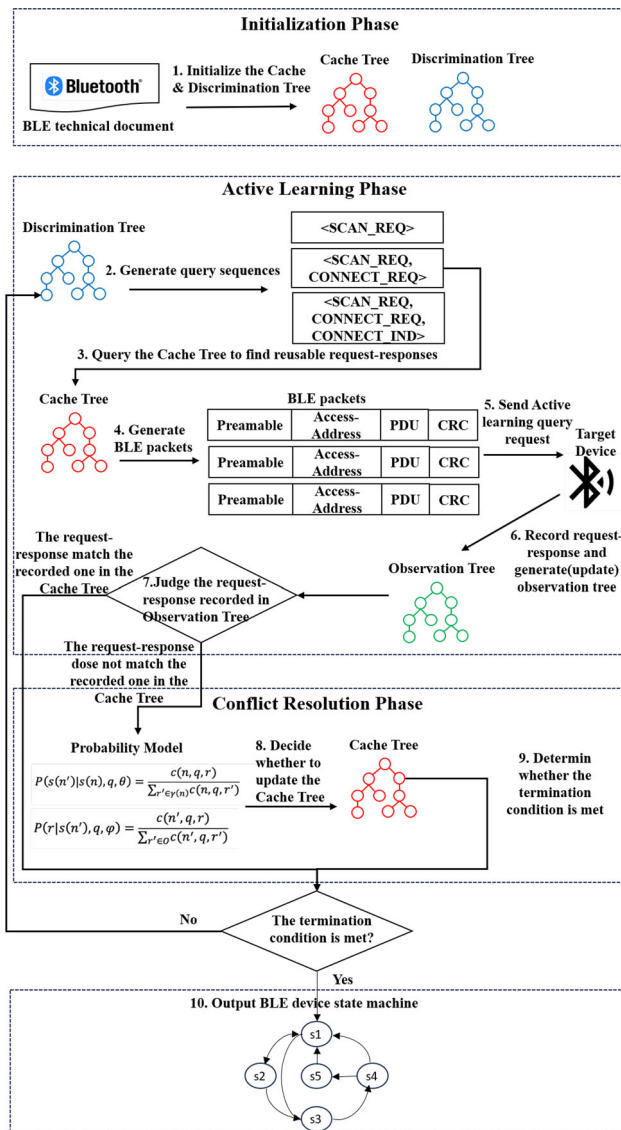


Fig. 2 The Conflict-Aware Active Learning Workflow

$$path(v) = \langle \rangle \cdot \prod_{(u, v) \in \pi(v)} in(v) \quad (1)$$

Where $\langle \rangle$ denotes an empty sequence, and $\prod$ denotes a concatenation.

In the conflict-aware active learning, $\forall v$, $path(v)$ represents a finite prefix of M , and $state(v)$ is the reached state through the path. The initial BLE state machine model $M = (S, s_0, I, O, \delta, \lambda)$ is encoded into the cache tree $T = (V, E)$ as follows:

- Construct $r \in V$ with $in(r) = out(r) = \emptyset$, $state(r) = \emptyset$.
- For each initial state $s \in s_0$, add a child node $v_s \in V$ under r with $in(v_s) = out(v_s) = \emptyset$ and $state(v_s) = s$.

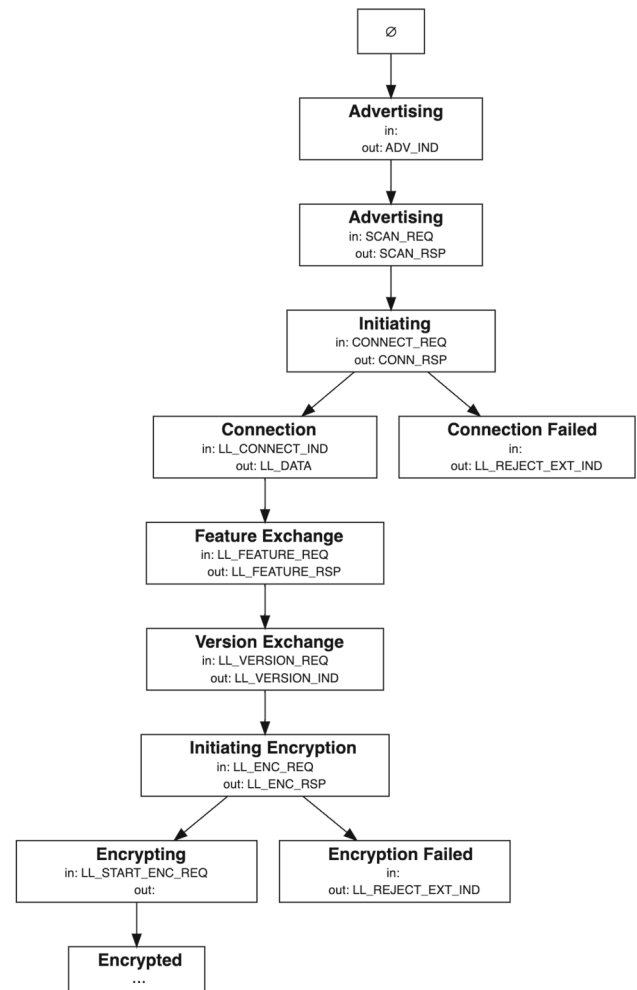


Fig. 3 A Cache Tree

- $\forall s \in S$ and $i \in I$, if $\delta(s, i) = s'$ and $\lambda(s, i) = o$, then an edge $(v_s, v_{s'}) \in E$ is established satisfying $in(v_{s'}) = \{i\}$, $out(v_{s'}) = \{o\}$, $state(v_{s'}) = s'$.

This iterative process encodes all states and transitions of M into T . As active learning progresses, the cache tree dynamically expands to capture the BLE device's complete states. A simplified illustration of the cache tree structure is provided in Figure 3.

4.3 Conflict-aware active learning

The Discrimination Tree, based on state transitions and request-response patterns specified in the BLE protocol, generates queries to optimally distinguish known states, enhancing active learning efficiency.

Definition 2 (Discrimination Tree) The Discrimination Tree is a hierarchical structure $T = (N, E)$. Each node $n \in N$ comprises two elements: U_n , a set containing the states asso-

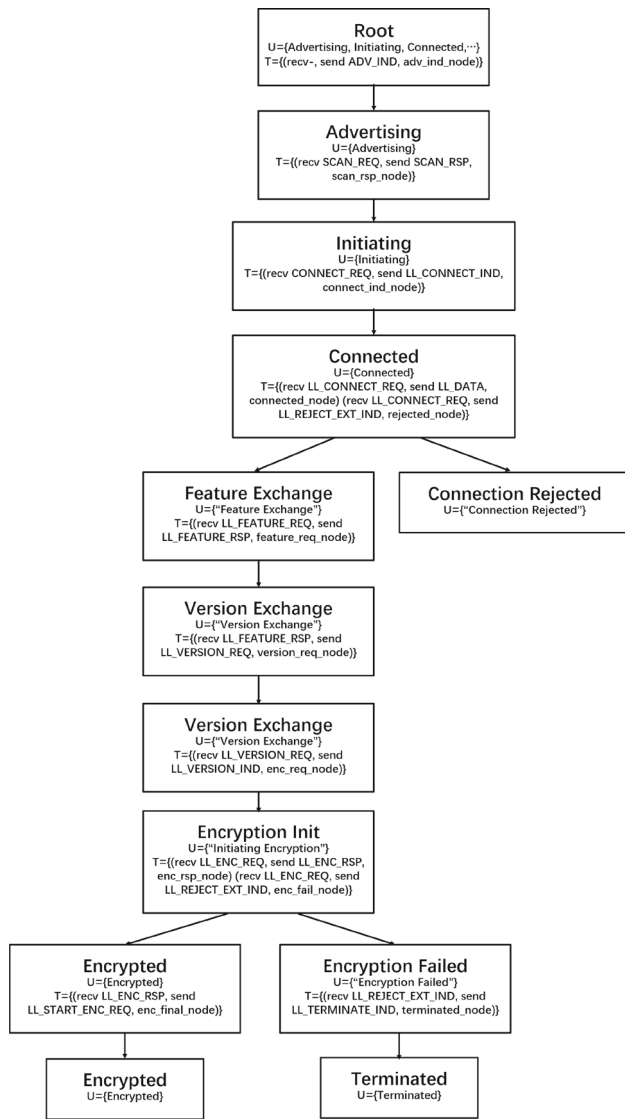


Fig. 4 A Discrimination Tree

ciated with that node, and T_n , a list of triples (q, r, n') , where q is the data sent to the target device, r is the corresponding response received, and n' is the child node derived from r . The edge set E consists of directed links, each corresponding to n' in the triple (q, r, n') from its parent node n . The root node, denoted as n_r , initially contains all possible states within U_{n_r} . Each leaf node n represents a single state, ensuring that $|U_n| = 1$.

Figure 4 depicts a discrimination tree. The process of generating query sequences through this tree involves traversing from the root node n_r . At each step, a triple (q, r, n') is arbitrarily selected from the current node, and q is appended to the ongoing query sequence. The traversal then moves to node n' , repeating this step until reaching a leaf node, thereby compositing a complete query sequence π . The algorithm then backtracks to explore alternative paths, iterating until

all possible query sequences $\Pi = \pi_1, \pi_2, \dots, \pi_m$ are generated, as detailed in Algorithm 1.

Algorithm 1 Generate query sequence using discrimination tree

Input: Discrimination Tree $T = (N, E)$, Root node n_r

Output: Query sequence set Π

```

1:  $\Pi \leftarrow \emptyset$  ▷ Initialize the query sequence set
2: procedure GENERATEQUERY( $n, sequence$ )
3:   if  $|U_n| = 1$  then ▷ Check if it is a leaf node
4:      $\Pi \leftarrow \Pi \cup \{sequence\}$  ▷ Add a sequence to the query sequence
5:   else
6:     for all  $(q, r, n') \in T_n$  do ▷ Traverse all triples of the current node
7:        $newSequence \leftarrow sequence + [q]$  ▷ Add new query to the sequence
8:       GENERATEQUERY( $n', newSequence$ ) ▷ Recursive call to process child nodes
9:   end for
10: end if
11: end procedure
  
```

The active learning starts by executing query sequences Π , each query sequence $\pi_i \in \Pi$ interacting with the cache tree $T_c = (V, E_c)$ to retrieve device responses. For each query q in π_i , the tree is searched for a node v where $in(v) = q$. If v is found, then q is executed probabilistically with probability p . If q is executed, the response r' updates the observation tree. If r' matches $out(v)$, the path through v is reused; otherwise, a conflict is detected and resolved, possibly adding a new node in T_c . If q is skipped, traversal proceeds from v . If no matching node exists, q is executed, r' is stored, and conflicts are resolved.

Once all sequences in Π have been processed, the observation and cache trees are updated, progressively refining the device's state machine. This iterative learning procedure is formally described in Algorithm 2.

4.4 Conflict resolution

In the BLE device active learning process, conflicts occur when the device's observed behavior deviates from the model's predictions, highlighting differences between the learned and actual state machine.

Definition 3 (Conflict) A conflict arises in the BLE state machine $M = (S, s_0, I, O, \delta, \lambda)$ if there exists $s \in S, q \in I$, and $r \in O$ satisfying:

- $\delta(s, q)$ indicates q is a valid query from state s .
- $r \neq \lambda(s, q)$, where $\lambda(s, q)$ represents the expected response.

Algorithm 2 Active Learning Query Algorithm

Require: Cache tree $T_c = (V, E_c)$, query sequences Π , execution probability p

Ensure: Updated cache tree T_c , observation tree T_o

```

1: procedure PROCESSQUERYSEQUENCES( $T_c, \Pi, p$ )
2:   for  $\pi_i \in \Pi$  do
3:     for  $q \in \pi_i$  do
4:        $v \leftarrow \text{FINDNODE}(T_c, q)$ 
5:       if  $v \neq \text{null}$  then
6:          $execute \leftarrow p$ 
7:         if  $execute$  then
8:            $r' \leftarrow \text{EXECUTEQUERY}(q)$ 
9:            $\text{UPDATEOBSTREE}(T_o, q, r')$ 
10:          if  $r' = \text{out}(v)$  then
11:            continue
12:          else
13:             $\text{RESOLVECONFLICT}(T_c, T_o, v, r')$ 
14:          end if
15:        else
16:          continue
17:        end if
18:      else
19:         $r' \leftarrow \text{EXECUTEQUERY}(q)$ 
20:         $\text{UPDATEOBSTREE}(T_o, q, r')$ 
21:         $\text{RESOLVECONFLICT}(T_c, T_o, v, r')$ 
22:      end if
23:    end for
24:  end for
25: end procedure

```

A conflict occurs when a query q from state s yields a response r differing from the expected $\lambda(s, q)$, indicating a model mismatch. Conflict resolution uses an observation tree and a probabilistic model.

Definition 4 (Observation Tree) An observation tree is a tree $O = (N, E, \Sigma, L, \gamma)$, where:

- N is the nodes set.
- $E \subseteq N \times \Sigma \times N$ is the directed edges set.
- Σ is a finite input queries set.
- L is a finite output responses set.
- $\gamma : N \rightarrow 2^L$ assigns each node $n \in N$ a subset of responses.

Each node $n \in N$ represents a state $s(n)$, and an edge $(n, q, n') \in E$ denotes a transition from $s(n)$ to $s(n')$ triggered by query $q \in \Sigma$. Counters $c(n, q, r)$ track the frequency of response r for q . These counters help quantify the distribution of responses and provide a means to assess uncertainties in the state machine model.

The observation tree is updated in two phases iteratively: recording observations and detecting conflicts. For a query sequence $\pi = \langle q_1, q_2, \dots, q_n \rangle \in \Sigma^*$ and response sequence $\rho = \langle r_1, r_2, \dots, r_n \rangle \in L^*$, for each $1 \leq i \leq n$, beginning with the root node n_r , if the edge $(n_{i-1}, q_i, n_i) \in E$ exists, its counter is incremented. If not, a new node n_i is created, and

Algorithm 3 Observation Tree Update Algorithm

Require: Observation tree T_o , query sequence q , response sequence r'

Ensure: Updated observation tree T'_o

```

1: procedure UPDATEOBSERVATIONTREE( $T_o, q, r'$ )
2:    $\text{RECORDOBSERVATION}(T_o, q, r')$ 
3:    $\text{CHECKFORCONFLICTS}(T_o, q, r')$ 
4: end procedure
5: procedure RECORDOBSERVATION( $T_o, q, r'$ )
6:    $n \leftarrow \text{root}(T_o)$ 
7:   for  $i = 1$  to  $\text{length}(q)$  do
8:     if  $\exists(n, q[i], n') \in E(T_o)$  then
9:        $c(n, q[i], r'[i]) \leftarrow c(n, q[i], r'[i]) + 1$ 
10:    else
11:       $n' \leftarrow \text{new\_node}()$ 
12:       $N(T_o) \leftarrow N(T_o) \cup \{n'\}$ ,  $E(T_o) \leftarrow E(T_o) \cup \{(n, q[i], n')\}$ 
13:       $\gamma(n') \leftarrow \{r'[i]\}$ ,  $c(n, q[i], r'[i]) \leftarrow 1$ 
14:    end if
15:     $n \leftarrow n'$ 
16:  end for
17: end procedure
18: procedure CHECKFORCONFLICTS( $T_o, q, r'$ )
19:    $n \leftarrow \text{root}(T_o)$ 
20:   for  $i = 1$  to  $\text{length}(q)$  do
21:     if  $\neg \exists(n, q[i], n') \in E(T_o) \vee r'[i] \notin \gamma(n')$  then
22:        $\text{HANDLECONFLICT}(n, q[i], r'[i])$ 
23:     end if
24:     if  $\exists(n, q[i], n') \in E(T_o)$  then
25:        $n \leftarrow n'$ 
26:     end if
27:   end for
28: end procedure

```

the edge (n_{i-1}, q_i, n_i) is added to E , initializing $\gamma(n_i) = \{r_i\}$ and $c(n_{i-1}, q_i, r_i) = 1$.

The conflict detection process works as follows: For $\forall q_i \in \pi$ at node n_{i-1} , the cache tree T_c is checked for a node v where $\text{path}(v) = \text{path}(n_{i-1})$ and $\text{in}(v) = \{q_i\}$. If $\exists v$, $r_i = \text{out}(v)$, no conflict occurs. If $\forall v$, $r_i \neq \text{out}(v)$, a conflict is detected.

Once conflicts are identified, the probabilistic model uses $c(n_{i-1}, q_i, r_i)$ from the observation tree to resolve discrepancies and update the cache tree. The update process is outlined in Algorithm 3.

In learning BLE device state machines, the probabilistic model evaluates discrepancies between actual device states and observed results, influenced by factors like wireless noise, signal fading, interference, or malfunctions. These factors can cause the analyst to receive BLE packets that don't reflect the actual state of the device. This decision-making process can be framed as a Partially Observable Markov Decision Process (POMDP), where unobservable states are represented by S , the adversary's actions by A , and observable responses by Ω .

Definition 5 (Probabilistic Model) The probabilistic model consists of two components: the state transition probability $P(s'|s, a, \theta)$, which estimates the likelihood of transitioning to state s' from state s after action a , and the observation

probability $P(o|s', a, \phi)$, which describes the likelihood of observation o after action a in state s' . In the BLE device state machine context, the action set A represents input queries I , and the observation set Ω corresponds to output responses O . The algorithm aims to estimate the parameters θ and ϕ using observed request-response pairs.

In the observation tree $O = (N, E, \Sigma, L, \gamma)$, each node $n \in N$ represents a state $s(n)$. The state transition probability $P(s(n')|s(n), q, \theta)$ is computed using $c(n', q, r)$, which records the occurrences of response r when query q is issued in state $s(n)$. The observation probability model $P(r|s(n'), q, \phi)$ is based on the frequency of observing response r at node n' when query q is issued.

$$P(s(n')|s(n), q, \theta) = \frac{c(n, q, r)}{\sum_{r' \in \gamma(n)} c(n, q, r')} \quad (2)$$

$$P(r|s(n'), q, \phi) = \frac{c(n', q, r)}{\sum_{r' \in O} c(n', q, r')} \quad (3)$$

To resolve conflicts, when an unseen response r^* is observed for query q in state $s(n)$, the potential new state $s(n^*)$ is evaluated by computing the combined probability p^* . If p^* exceeds a threshold τ , $s(n^*)$ is considered a new state and added to the state machine, with the counts updated accordingly. Otherwise, r^* is treated as noise, and only the counts are adjusted. This process also involves monitoring p^* over multiple observations to identify trends, ensuring robust distinction between genuine states and noise. Details of the conflict resolution are elaborated in Algorithm 4.

$$\begin{aligned} p^* &= P(r^*|s(n^*), q, \phi) \times P(s(n^*)|s(n), q, \theta) \\ &= \frac{c(n^*, q, r^*)}{\sum_{r' \in O} c(n^*, q, r')} \times \frac{c(n, q, r^*)}{\sum_{r' \in \gamma(n)} c(n, q, r')} \end{aligned} \quad (4)$$

In this work, we assume that state transitions and observations are conditionally independent. This assumption generally holds in cases with independent or low-correlation noise. However, in scenarios with highly correlated noise (e.g., when sequential data is significantly influenced by adjacent time steps), this assumption may no longer hold. Therefore, future work could explore incorporating noise correlation modeling to improve the model's adaptability. Additionally, further modeling adjustments may be needed to enhance the model's robustness in highly correlated noise conditions.

5 State coverage guided gray-box fuzz testing

The state coverage guided gray-box fuzzing framework, depicted in Figure 5, consists of three key processes: ini-

Algorithm 4 Conflict Resolution Algorithm

Require: Cache tree T_c , observation tree T_o , cache tree node v , response r' , threshold τ

Ensure: Updated cache tree T_c , updated observation tree T_o

```

1: procedure RESOLVECONFLICT( $T_c, T_o, v, r'$ )
2:    $n \leftarrow \text{FINDOBSNODE}(T_o, \text{path}(v))$ 
3:   if  $n = \text{null}$  then
4:      $\text{CREATENEWBRANCH}(T_o, \text{path}(v), r')$ 
5:   else
6:      $p^* \leftarrow \text{COMPUTE OBS PROB}(n, q, r') \times \text{COMPUTE TRANSITION PROB}(n, q, r')$ 
7:     if  $p^* \geq \tau$  then
8:        $n^* \leftarrow \text{new\_node}()$ ,  $N(T_o) \leftarrow N(T_o) \cup \{n^*\}$ ,  $E(T_o) \leftarrow E(T_o) \cup \{(n, \text{in}(v), n^*)\}$ 
9:        $\gamma(n^*) \leftarrow \{r'\}$ ,  $c(n, \text{in}(v), r') \leftarrow 1$ 
10:       $\text{UPDATECACHETREE}(T_c, \text{path}(v), \text{in}(v), r')$ 
11:     else
12:        $c(n, \text{in}(v), r') \leftarrow c(n, \text{in}(v), r') + 1$ 
13:     end if
14:   end if
15: end procedure
16: procedure COMPUTE TRANSITION PROB( $n, q, r'$ )
17:    $\text{denominator} \leftarrow \sum_{r'' \in \gamma(n)} c(n, q, r'')$ 
18:   return  $\frac{c(n, q, r')}{\text{denominator}}$  if  $\text{denominator} > 0$  else 0
19: end procedure

```

tial test case generation, test case scheduling optimization, and test execution with feedback analysis. Its core lies in a state-coverage-oriented test case generation algorithm and an evolutionary algorithm-based scheduling optimization strategy.

(1) Test Case Generation. The method constructs an initial high-coverage test case pool using a state-coverage-oriented algorithm based on the BLE device state machine. By integrating state machine information and BLE protocol semantics, it enhances the probability of triggering critical state transitions. The algorithm generates BLE protocol packets as initial test cases, effectively exploring key execution paths in the device state machine.

(2) Test Case Scheduling Optimization. The generated test cases are optimized through an evolutionary algorithm. This module encodes mutation strategies for mutable BLE protocol packet fields and the scheduling order of test case execution into a probability vector representing individuals in the evolutionary algorithm's population. Each generation of the evolutionary iteration produces new test case sets through packet mutations and sequence reordering based on the probability vector.

(3) Test Execution and Feedback Analysis. The optimized test case set is executed by the testing engine, which interacts with the target BLE device while monitoring and recording its responses. Metrics such as state coverage, anomaly-triggering capability, and execution efficiency are used to calculate a comprehensive fitness score. Feedback from these metrics guides the next iteration of evolutionary optimization.

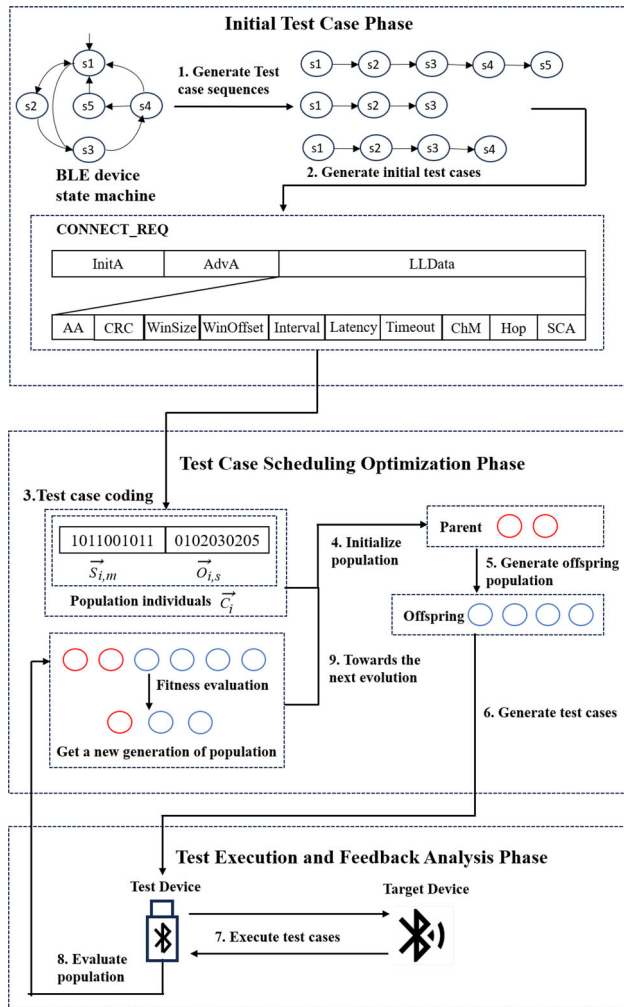


Fig. 5 State Coverage Guided Grey-box Fuzzing Method

The aforementioned closed-loop process of test case generation, scheduling, execution, and feedback analysis iteratively enhances test case quality. The method produces test case sets with high coverage and strong defect detection capabilities, enabling a thorough evaluation of the security and reliability of BLE devices.

5.1 Test case generation

This section outlines the systematic generation of test case packet sequences T based on the BLE device state machine M . The algorithm takes as input the state machine model M and the initial state s_0 . It initializes a visited state set $V = s_0$ and an empty test case set $T = \emptyset$. The detailed procedure is described in Algorithm 5.

For each visited state $s \in V$, the algorithm performs the following steps:

Algorithm 5 State-Coverage-guided Test Case Generation Algorithm

Input: State machine model M , initial state s_0

Output: Test case data packet sequence set T

```

1: function GENERATE_TEST_CASES( $M, s_0$ )
2:    $V \leftarrow \{s_0\}$  ▷ Visited state collection
3:    $T \leftarrow \emptyset$  ▷ Test case collection
4:    $P(s_0) \leftarrow \{\epsilon\}$  ▷  $\epsilon$  Indicates an empty path
5:   while  $\exists s \in V$  do
6:     for  $p \in P(s)$  do
7:        $T(s, p) \leftarrow \emptyset$ 
8:       for  $d \in D(p)$  and CHECK_CONSTRAINTS( $d$ ) do
9:          $T(s, p) \leftarrow T(s, p) \cup \{p \oplus \langle d \rangle\}$ 
10:      end for
11:       $T \leftarrow T \cup T(s, p)$ 
12:    end for
13:    for  $s' \notin V$  and CAN_REACH_FROM_CURRENT_STATE( $M, s, s'$ ) do
14:       $V \leftarrow V \cup \{s'\}$ 
15:      for  $p \in P(s)$  do
16:         $P(s') \leftarrow P(s') \cup \{p \oplus \langle t \rangle\}$ 
17:      end for
18:    end for
19:  end while
20:  return  $T$ 
21: end function
22: function CAN_REACH_FROM_CURRENT_STATE( $M, s, s'$ )
23:  for  $t \in \Sigma(M)$  do
24:    if  $\delta(s, t) = s'$  then
25:      return True
26:    end if
27:  end for
28:  return False
29: end function

```

1. Identify all paths $P(s)$ that can reach state s , where $P(s_0) = \epsilon$ (the empty path).
2. For each path $p \in P(s)$, construct a set of test case packet sequences $T(s, p)$. Let $D(p)$ represent the mutation space of the last packet in p , containing all possible field value combinations. A mutation value $d \in D(p)$ defines a complete packet instance with specific field values. The sequence $p \oplus \langle d \rangle$ is formed by appending d to p . The set $T(s, p) = \bigcup_{d \in D(p)} \{p \oplus \langle d \rangle\}$ is the union of all sequences.
3. Use heuristic mutation operators M to generate $D(p)$, considering protocol-specific constraints such as data type, value range, and semantic dependencies.
4. Merge all test cases in $T(s, p)$ into the final test case set T .
5. For each successor state $s' \notin V$, the algorithm adds s' to the set V and adds (p, t) to $P(s')$, where t is the data packet that transitions s to s' .

The algorithm iterates these steps until all reachable states are visited, producing the final test case set T .

For example, the mutation of a pairing request packet *CONNECT_REQ* demonstrates the use of mutation oper-

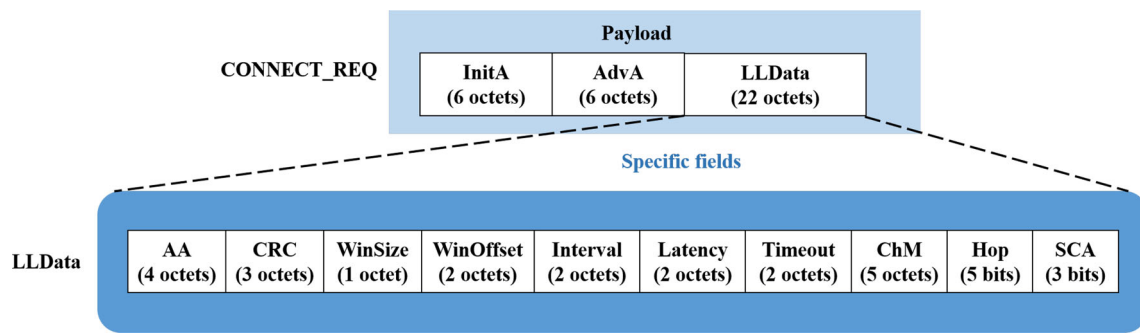


Fig. 6 **CONNECT_REQ** Packet

ators. BLE packet mutations must adhere to protocol-defined constraints, including data type, value range, and semantic dependencies. Mutation operators M are specifically designed to alter each field of the packet according to these constraints. The structure of the pairing request packet is illustrated in Figure 6.

The packet mutation process begins with the first field of the **CONNECT_REQ**, *InitA* (Initiator Address), and recursively explores subsequent PDU fields; the *InitA* field, a 6-byte device address, must comply with BLE specifications requiring the first 5 bytes to represent the device identifier, the last byte to be a random number, and prohibiting addresses consisting entirely of zeros or ones or conflicting with reserved addresses; based on these constraints, the algorithm defines mutation operators for *InitA*, including random generation of valid addresses, selection from a known database, mutation of known addresses through bit flipping or byte swapping, and generation of boundary condition addresses such as all zeros, all ones, or broadcast addresses.

For subsequent PDU fields, a similar strategy is employed. For example, the *AA* field, mandated by the BLE specification to be fixed at 0x8E89BED6, is mutated into invalid values such as 0x8E89BED7 or random 4-byte values to assess the handling of erroneous packets. For integer-type fields like *WinSize* and *Interval*, the algorithm generates random values within valid ranges, boundary values (e.g., minimum, maximum, zero), and invalid values outside the specified range. For multi-subfield fields like *Hop* and *SCA*, which share a single byte—with *Hop* defining the data channel hopping sequence (range 0-31) and *SCA* representing clock accuracy (range 0-7)—the algorithm produces combinations adhering to both constraints, while also introducing boundary and invalid values to strengthen the testing process.

After generating a complete mutated **CONNECT_REQ**, the algorithm verifies compliance with constraints such as packet length and CRC, adding valid packets to the test set. Otherwise it backtracks to explore alternative mutation values.

5.2 Test case scheduling optimization

To further enhance testing efficiency and effectiveness, this section proposes a test case scheduling optimization method based on evolutionary algorithms. For an initial test case set $T = \{t_1, t_2, \dots, t_m\}$, where each t_i represents a BLE packet sequence, the algorithm aims to optimize each t_i through packet field mutation and structural variation operations to improve testing efficiency and the ability to trigger abnormal behavior.

Specifically, the optimization objectives include three aspects:

1. **Maximizing Test Case Diversity:** Optimizing the diversity of packet field values in the test case t'_i , which includes different field types, value ranges, and boundary conditions. The diversity of t'_i is denoted as $d(t'_i)$. A higher diversity results in a higher score.
2. **Maximizing the Ability to Trigger Abnormal Behavior:** Optimizing the proportion of test cases in the optimized subset that trigger abnormal behaviors, such as device restarts, connection interruptions, response timeouts, and unexpected responses. This ability is denoted as $a(t'_i)$.
3. **Execution Time:** The execution time of a single optimized test case t'_i , denoted as $t(t'_i)$.

Considering these three objectives, the algorithm designs a weighted fitness function:

$$F(t'_i) = w_1 \cdot d(t'_i) + w_2 \cdot a(t'_i) - w_3 \cdot \frac{t(t'_i)}{t_{\max}} \quad (5)$$

The weight coefficients w_1 , w_2 and w_3 correspond to the three objectives. They are initially assigned random values while ensuring that their sum equals 1 ($w_1 + w_2 + w_3 = 1$). During each mutation step, these coefficients are iteratively updated with new random values that maintain this constraint, gradually stabilizing over time.

For each initial test case $t_i \in T$, the algorithm represents its optimization strategy as a hybrid encoded vector $\vec{c}_i$.

$$\vec{c}_i = (\vec{s}_{i,m}, \vec{o}_{i,s}) \quad (6)$$

Here, $\vec{s}_{i,m}$ is the packet field mutation strategy vector for t_i , and $\vec{o}_{i,s}$ is the packet sequence encoding vector, representing the execution order of packets. $\vec{s}_{i,m}$ is detailed as follows, where n_i is the total number of packets in t_i , $n_{i,j}$ is the number of mutable fields in the j -th packet of t_i , and $s_m^{i,j,k} \in 0, 1$ indicates whether the k -th field of the j -th packet is mutated:

$$\vec{s}_{i,m} = (s_m^{(i,1,1)}, s_m^{(i,1,2)}, \dots, s_m^{(i,j,k)}, \dots, s_m^{(i,n_i,n_{i,j})}) \quad (7)$$

The algorithm's goal is to search for the optimal encoding $\vec{c}_i^*$ in the encoding space of $\vec{c}_i$ using an evolutionary algorithm to maximize the fitness function $F(t'_i)$. The corresponding t'_i is the desired high-quality optimized test case.

The algorithm employs the following evolutionary strategy to optimize the encoding for each t_i :

1. Initialize a population $P_i = \{\vec{c}_{i,1}, \vec{c}_{i,2}, \dots, \vec{c}_{i,N}\}$, where each $\vec{c}_{i,j}$ is a randomly initialized hybrid encoded vector.
2. Decode each $\vec{c}_{i,j}$ to obtain $t'_{i,j}$ and calculate its fitness $F(t'_{i,j})$.
3. Select the individuals with higher fitness from P_i as the parent generation P_i^{parent} .
4. Perform mutation operations on the encoded layer of P_i^{parent} : retain $\vec{o}_{i,s}$, and independently flip each bit in $\vec{s}_{i,m}$ with a flip probability p_m , generating a new offspring population $P_i^{offspring}$.
5. Apply mutation operations to $P_i^{offspring}$: mutate the necessary parts of $\vec{s}_{i,m}$ using mutation operators, and perform position swaps, insertions, etc., on $\vec{o}_{i,s}$.
6. Merge $P_i^{offspring}$ into P_i , retaining the top N individuals based on fitness as the new population, completing one evolution cycle.
7. Repeat steps (2)–(6) until the termination condition is met. Output the optimal encoding $\vec{c}_i^*$ and the corresponding optimized test case t'_i .

6 Experiments

To assess the effectiveness of the proposed conflict-aware active learning algorithm in modeling BLE device state machines, experiments examined query efficiency, resilience to noise, parameter sensitivity, and practical applicability. Performance comparisons with existing approaches highlight its advantages, while an in-depth evaluation of noise

Algorithm 6 Test Case Optimization Based on Evolutionary Algorithm

Input: Initial test case set $T = \{t_1, t_2, \dots, t_m\}$
Output: Optimized test case set $T' = \{t'_1, t'_2, \dots, t'_m\}$

```

1: function EVOLUTIONARYOPTIMIZATION( $T, max\_iter, N$ )
2:    $T' \leftarrow \emptyset$   $\triangleright$  Store optimized test case collection
3:   for  $t \in T$  do  $\triangleright$  Optimize each initial test case
4:      $P \leftarrow \text{INITIALIZEPOPULATION}(t, N)$   $\triangleright$  Initialize the population
5:     for  $iter = 1 \dots max\_iter$  do  $\triangleright$  Iterative Optimization
6:        $\text{EVALUATEFITNESS}(P)$   $\triangleright$  Evaluate individual fitness
7:        $P_{parent} \leftarrow \text{SELECTPARENTS}(P)$   $\triangleright$  Select Parent
8:        $P_{offspring} \leftarrow \text{MUTATE}(P_{parent})$   $\triangleright$  Mutate to generate offspring
9:        $P \leftarrow \text{SURVIVALSELECTION}(P, P_{offspring}, N)$   $\triangleright$  Select to form new populations
10:    end for
11:     $c_{opt}, t_{opt} \leftarrow \text{GETBESTINDIVIDUAL}(P)$   $\triangleright$  Get the best individual
12:     $T' \leftarrow T' \cup \{t_{opt}\}$   $\triangleright$  Add the optimized test cases to the collection
13:  end for
14:  return  $T'$ 
15: end function

```

impact, parameter configurations, and state machine complexity provides insights into its robustness and scalability.

6.1 Experimental setup

To thoroughly assess the performance of the proposed conflict-aware active learning algorithm, a comparison was made with two benchmark algorithms: Lstar [15], TTT [9], and L2Fuzz [14]. The experiments involved using a Nordic nRF52840 as the BLE central device and an ESP32 as the BLE peripheral device under test. A simulated environment was created in a 20-square-meter laboratory to replicate real-world wireless interference. This setup included 10 BLE devices, two 2.4GHz WiFi routers, and three wireless mice, ensuring continuous interference across the BLE and 2.4GHz spectrums.

The evaluation also involved several representative BLE devices, including ESP32, CC2640R2, and Nordic's nRF52832 and nRF52840, which vary in functionality and protocol complexity. The BLE hardware devices used in the experiment are shown in Table 1, and the software development environment is shown in Table 2. These devices, equipped with robust SDKs and debugging interfaces, provided a thorough test of the algorithm's adaptability, scalability, and efficiency. The analysis emphasized state coverage, vulnerability detection, test case generation efficiency, and vulnerability detection speed to identify optimization opportunities.

Table 1 BLE Device Specifications

No.	Manufacturer	Model	BLE Version
1	Espressif Systems	ESP32-WROOM-32D	4.2
2	Texas Instruments	CC2640R2	5
3	Nordic Semiconductor	nRF52840 BLE dongle	5.2
4	Nordic Semiconductor	nRF52 DK	5.4

Table 2 System Development Environment

Category	Details
Operating System	Ubuntu 22.04
Development Environment	Python 2.7, Python 3.9, VS Code
Firmware Version	esp-idf v5.2 SimpleLink CC2640R2 SDK v5.30.0.3 Zephyr-bluetooth-peripheral nRF5 SDK v17.1.0

6.2 Experimental results

6.2.1 BLE state machine construction

Testing was conducted on three BLE state machines of increasing protocol complexity: Simple (basic processes like broadcasting and data transmission), Medium (including parameter updates and secure connections), and Complex (encompassing broadcast distinctions, passive scanning, and low-power modes). Noise scenarios were categorized into three types—Random Noise, Burst Noise, and Continuous Noise—with error rates of 5%, 10%, and 20% representing low, medium, and high intensities. Performance metrics included the number of queries (efficiency), the number of states in the final model (detail), and convergence time (speed).

To evaluate baseline performance, the algorithm was tested in various simulated interference environments. As shown in Figure 7, it enhanced BLE state machine modeling by increasing state discovery by 30%, reducing query overhead by 25%, and accelerating convergence by 50%, while dynamically resolving state transition conflicts. This experiment serves as a benchmark for assessing robustness under varying levels of interference and protocol complexity. Across different noise conditions, our approach demonstrated superior efficiency, achieving the fewest queries, the highest state discovery rate, and the shortest convergence time.

As the number of BLE states increases, the runtime overhead of the four benchmark approaches grows at different rates (Figure 7(d)). Conflict-Aware shows a moderate increase from 26 minutes (8 states) to 66 minutes (24 states), indicating manageable scaling. Lstar rises from 52 to 123 minutes, reflecting a heavier computational burden, while TTT sees the steepest climb from 43 to 141 minutes, high-

lighting high computational demands. Although all methods face increased overhead, Conflict-Aware scales more efficiently, making it better suited for complex BLE protocols, whereas Lstar, TTT, and L2Fuzz struggle with the growing runtime.

To analyze noise effects, three types of noise (random, burst, and continuous) were introduced at low, medium, and high intensities. Figure 8 shows that noise reduced query efficiency, state discovery, and convergence speed, with continuous noise having the greatest impact and random noise the least. Performance degradation was most significant under high-intensity continuous noise, but the algorithm remained robust even under these adverse conditions.

The effect of algorithm parameters, particularly τ – which controls the threshold for updating the probabilistic model – was also examined. Five values of τ (0.1, 0.3, 0.5, 0.7, 0.9) were tested to evaluate query efficiency and convergence time. Results, presented in Figure 9, highlight the influence of τ on performance.

6.2.2 State coverage guided grey-box fuzzing

To evaluate the proposed State Coverage Guided Grey-box Fuzzing (SCDGF) framework, we benchmarked it against three fuzzing methods: Random Mutation Fuzzing (RMF), a black-box approach generating random BLE packets; State Machine Model-based Fuzzing (SMMF), a white-box method using pre-designed state models; and Heuristic Rule-based Mutation Fuzzing (HRMF), a gray-box method leveraging predefined BLE rules. These benchmarks cover black-box, white-box, and gray-box approaches for a balanced comparison. We also compared the proposed method with industrial tools like SweenTooth [7], BLE-Fuzz [15], BLE-Diff [10], and BrackTooth [6] to assess its real-world effectiveness in detecting BLE protocol vulnerabilities.

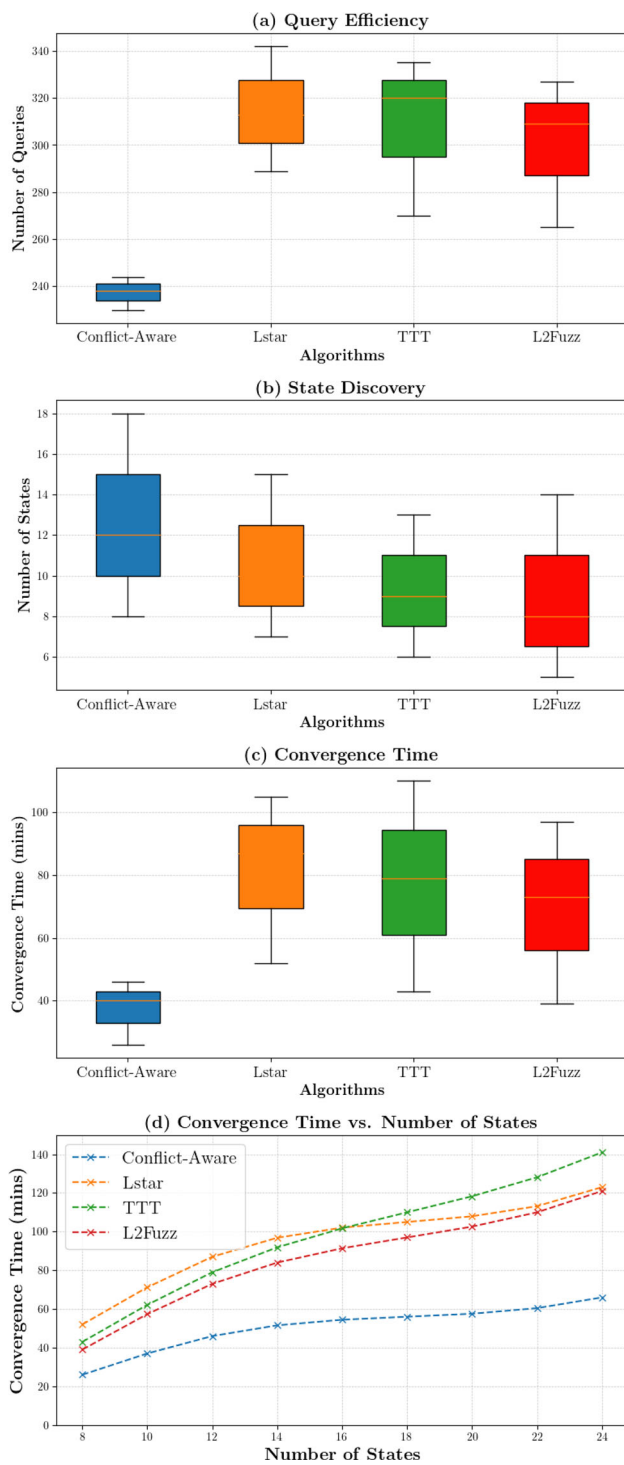


Fig. 7 The Conflict-Aware Active Learning Algorithm Baseline

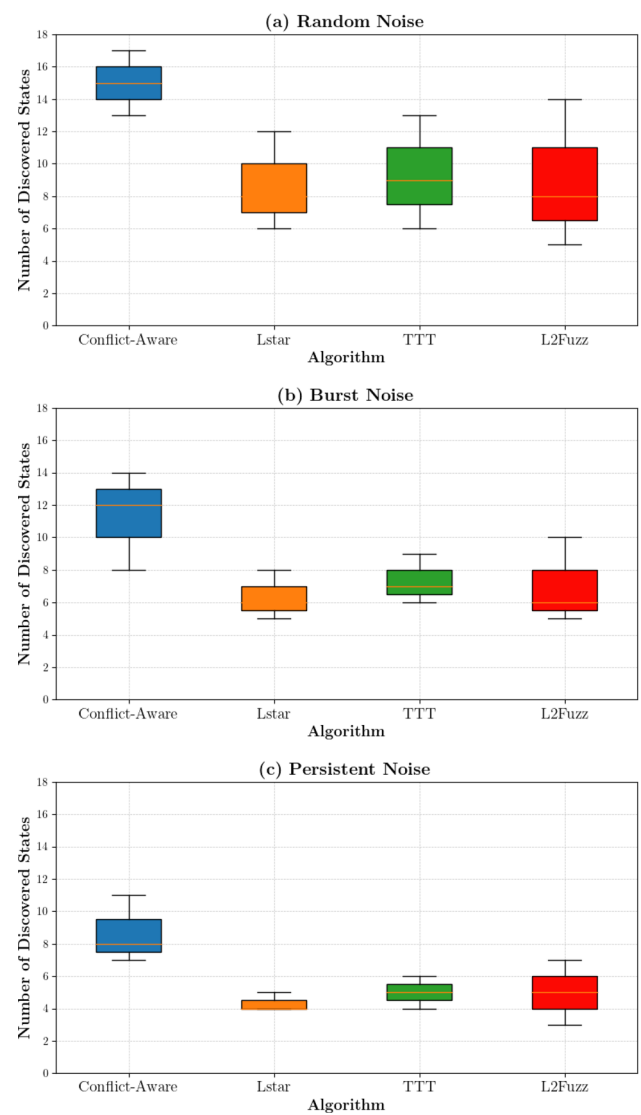


Fig. 8 Performance of Algorithm in Noisy Environments

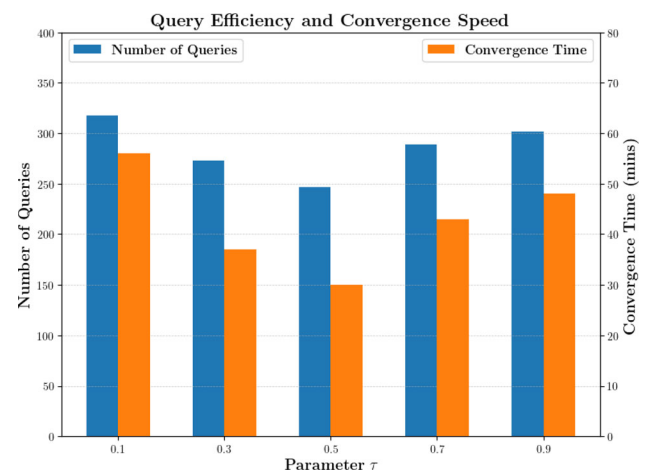


Fig. 9 Effect of Algorithm Parameter τ on Performance

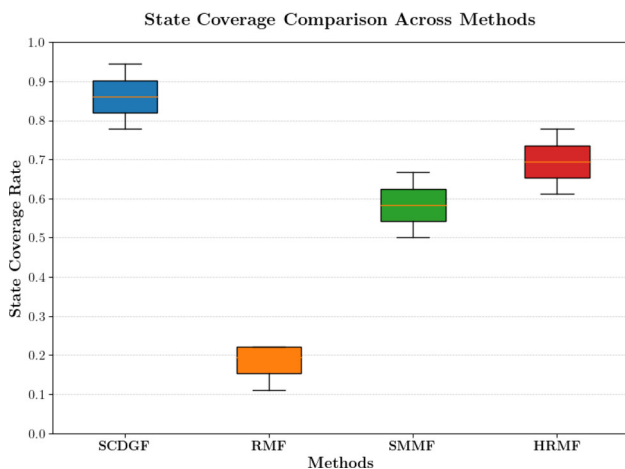


Fig. 10 State Coverage Comparison Across Methods

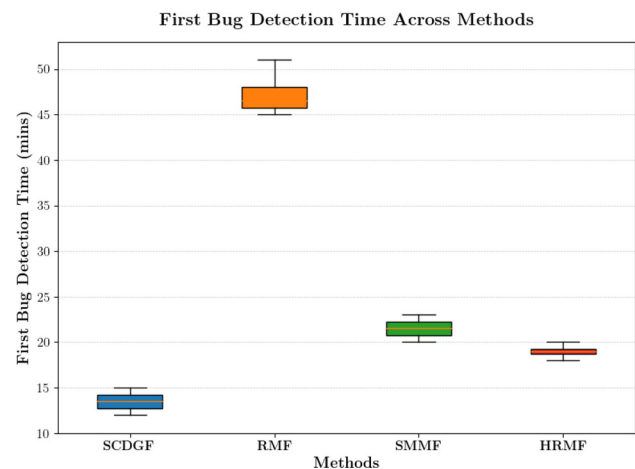


Fig. 12 First Bug Detection Time Across Methods

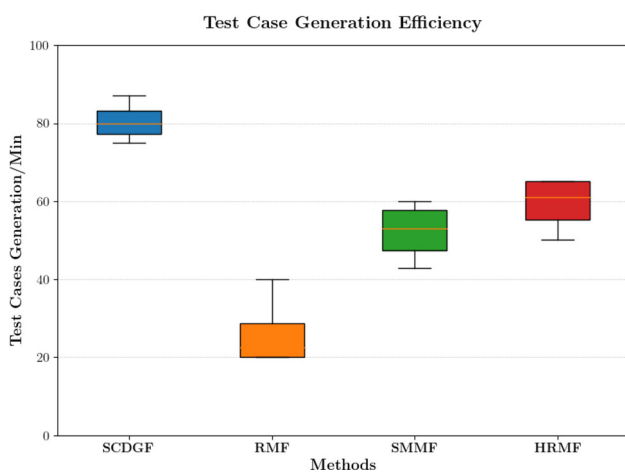


Fig. 11 Test Cases Generation Efficiency Comparison

Figure 10 shows that SCDGF achieves significantly higher state coverage rate across all test devices, demonstrating its effectiveness in leveraging state machine and protocol semantics to explore the device state space more thoroughly.

Figure 11 demonstrates the superior efficiency of SCDGF in generating test cases, consistently producing high-quality cases at a faster rate than the baseline methods, with an average increase of nearly 30% over the mean number generated by other benchmark approaches.

Figure 12 shows that SCDGF detects the first vulnerability with fewer test cases and in less time, reducing the average detection time by nearly 25% compared to other benchmark approaches. This demonstrates its superior efficiency in vulnerability detection among all baseline methods.

Table 3 presents a summary of the fuzz testing results on the tested BLE devices (ESP32, CC2640R2, nRF52840 and Zephyr). The prefix "V" indicates a discovered vulnerability, whereas "A" denotes an anomalous behavior that, while deviating from the expected operation defined in the Blue-

tooth Core Specification, does not qualify as a vulnerability. In total, the fuzzer identified 16 publicly disclosed vulnerabilities and detected two instances of unexpected behavior on the evaluated devices nRF52840 and ESP32.

Table 4 compares SCDGF with three benchmark BLE fuzzers SweenTooth, BLE-Fuzz, BLE-Diff and BrackTooth, revealing that SCDGF not only detects all vulnerabilities (the V1-V16 in Table 3) found by these tools but also uncovers new ones (the A1 and A2 in Table 3). Therefore, the evaluations of state coverage, test case generation, and vulnerability detection demonstrate the proposed SCDGF's superiority in enhancing BLE device testing capability on identifying vulnerabilities.

In summary, our approach uses state coverage-guided test case generation and evolutionary optimization to efficiently trigger deep behavioral states missed by traditional fuzzers. Unlike prior methods with predefined state transitions, it dynamically explores undiscovered protocol states, improving coverage and exposing new vulnerabilities. By integrating protocol semantics, it ensures valid mutations while maximizing fault detection. Beyond packet-level anomalies, it identifies state inconsistencies, logic errors, and exception handling flaws, enabling the discovery of both known and previously unreported BLE vulnerabilities.

7 Conclusion

In this paper, we present a novel gray-box fuzz testing method to address the limitations of existing BLE device security testing approaches. By integrating a conflict-aware active learning framework with data structures such as cache trees, distinction trees, observation trees, and probabilistic models, the method enables efficient construction of BLE device state machines. Furthermore, the state coverage-guided fuzz testing strategy, leveraging BLE protocol semantics,

Table 3 Summary of Bluetooth Vulnerabilities and Anomalies Identified in Tested BLE Devices

Anomaly	CVE ID	Device	Impact	Validation Method
V1 Feature Page Execution	CVE-2021-28139	ESP32	ACE / Deadlock	Send crafted packets and observe crashes or unexpected behavior.
V2 Duplicated IOCAP	CVE-2021-28136	ESP32	Crash (Reboot)	Repeatedly send duplicated IOCAP packets and check for system instability.
V3 Feature Res. Flooding	CVE-2021-28135	ESP32	Crash (Reboot)	Overload with feature resolution requests and monitor for failures.
V4 Invalid Public Key	CVE-2021-28138	ESP32	Crash (Reboot)	Send an invalid key and check for crashes.
V5 Feature Req. Ping-Pong	CVE-2021-28137	ESP32	Crash (Reboot)	Continuously exchange feature requests and verify stability.
V6 Invalid Connection Request	CVE-2019-19193	CC2640R2	Crash (DoS)	Send malformed connection requests and check for service denial.
V7 Key Exchange Bypass	CVE-2020-13593	CC2640R2	Possible MITM	Simulate MITM attacks during key exchange and verify acceptance of invalid keys.
V8 Out-of-Bounds Write	CVE-2018-16986	CC2640R2	Arbitrary Code Execution	Attempt buffer overflow and monitor execution anomalies.
V9 OAD RCE	CVE-2018-7080	CC2640R2	Remote Code Execution	Trigger OAD (Over-the-Air Download) and validate unauthorized execution.
V10 Public Key Crash	CVE-2019-17520	CC2640R2	Crash (DoS)	Send a corrupted public key and monitor device stability.
V11 Invalid Channel Map	CVE-2020-10069	Zephyr	Crash (DoS)	Use an invalid channel map and check for service disruption.
V12 Invalid Channel Map	CVE-2021-3433	Zephyr	Deadlock	Assess device response to an invalid channel map.
V13 Stack Buffer Overflow	CVE-2021-3434	Zephyr	Crash (Reboot)	Send excessive data to trigger buffer overflow.
V14 ECDSA Timing Side-Channel	CVE-2021-29415	nRF52840	Private Key Theft	Measure response time variations during key generation.
V15 File Permission Flaw	CVE-2023-46870	nRF52840	Arbitrary Code Execution	Attempt unauthorized file access and test execution privileges.
V16 Unsecure Pairing	CVE-2023-24023	nRF52840	Encryption Key Leakage	Pair with unsecure connections and monitor key exposure.
A1 CONNECT_IND hopIncrement < 5	N/A	nRF52840	Non-Compliance	Send CONNECT_IND packets with low hopIncrement and validate behavior.
A2 LL_LENGTH_REQ Rx Octets = 0	N/A	ESP32	Non-Compliance	Test response to zero-length Rx octets in LL_LENGTH_REQ requests.

Table 4 Comparison of Different Fuzz Testing Methods on BLE Devices

Metric	Device	SweynTooth	BLE-fuzz	BLE-diff	BrackTooth	SCDGF
Effective Vulnerabilities	ESP32	3	4	4	5	6
	CC2640R2	3	3	5	3	5
	nRF52840	3	2	2	3	4
	Zephyr	3	3	3	3	3
Detection Time	ESP32	2h 13min	2h 29min	1h 53min	2h 15min	1h 4min
	CC2640R2	1h 57min	2h 16min	2h 3min	1h 55min	1h 14min
	nRF52840	54min	1h 24min	1h 3min	58min	34min
	Zephyr	1h 32min	1h 54min	1h 12min	1h 36min	49min

The bold words show the indicators achieved by our method, which are different from those of other methods

state coverage-guided test case generation, and evolutionary optimization, effectively uncovers deep device behaviors. Experimental results demonstrate the proposed method's superiority in testing efficiency and vulnerability discovery in practical BLE scenarios. The implementation of a prototype system, which successfully identified multiple representative vulnerabilities on real BLE devices, validates the practicality and efficacy of the approach in advancing automated BLE security testing.

Appendix A: Symbol Notation

The declarations of all the symbols used in the paper are listed in Table 5.

Table 5 The Common Symbols in This Work

Notation	Description
S	BLE protocol states
s_0	Initial state
I, O	Input / output alphabet
M	BLE state machine model
$\theta : S \times I \rightarrow S$	Transition function
$\lambda : S \times I \rightarrow O$	Output function
T	Cache / Observation / Discrimination Tree
V (or N), E	Nodes / edges of T
$in(v), out(v)$	Input / output packets of v
$\langle \rangle$	Empty packet sequence
Π	Concatenation symbol
$\pi(v)$	Edges from root r to v
$path(v)$	Input packet sequence from r to v
$state(v)$	State of node v
(q, r, n')	Received data, response, child node
U_n, T_n	States set / triples list of n
Σ, L	Input / output query set
$\gamma(n)$	Mapping from n to responses
$c(n, q, r)$	Counter recording $\langle q, r \rangle$ at n
A	Adversary's query actions
Ω	Observation space
$P(s(n') s(n), q, \theta)$	State transition probability
$P(r s(n'), q, \phi)$	Observation probability
p^*	Combined probability model
$T(s, p)$	Test case packet sequences
$D(p)$	Mutation space of last packet in p
$F(t'_i)$	Weighted fitness function
$\vec{s}_{i,m}$	Packet field mutation strategy
$\vec{o}_{i,s}$	Packet sequence encoding
$\vec{c}_i$	Hybrid vector of $\vec{s}_{i,m}, \vec{o}_{i,s}$
P_i	Test case population
$P_i^{parent}, P_i^{offspring}$	Parent / offspring population

Appendix B: A BLE Fuzz Testing Example

In this section, we present real-world vulnerabilities identified using the proposed method, including the process of detecting anomaly A2 in Table 3. As illustrated in Figure 13(a), after sending the pairing_req message, the fuzzer applies a mutation algorithm to generate an *LL_LENGTH_REQ* request and transmits the malformed packet to the BLE device.

When the Max Rx Octets field, representing the maximum number of bytes the requester can receive (ranging from 27 to 251), in the *LL_LENGTH_REQ* request is set to 0, the ESP32 responds with an anomalous *LL_REJECT_EXT_IND* packet, deviating from the expected behavior defined by the Mealy state machine. Consequently, this scenario is

6. Garbelini, M.E., Bedi, V., Chattopadhyay, S., Sun, S., Kurniawan, E.: BrakTooth: causing havoc on Bluetooth link manager via directed fuzzing. *USENIX Secur. Symp.* 2022, 1025–1042 (2022). <https://www.usenix.org/conference/usenixsecurity22/presentation/garbelini>
7. Garbelini, M.E., Wang, C., Chattopadhyay, S., Sumei, S., Kurniawan, E.: SweynTooth: unleashing mayhem over Bluetooth low energy. *USENIX Annu. Tech. Conf.* 2020, 911–925 (2020). <https://www.usenix.org/conference/atc20/presentation/garbelini>
8. IW, F.: SAGE: whitebox fuzzing for security testing. *SAGE* 10(1) (2012). <https://doi.org/10.1145/2090147.2094081>
9. Isberner, M., Howar, F.M., Steffen, B.: The TTT Algorithm: A Redundancy-Free Approach to Active Automata Learning. *Runtime Verif.* (2014). https://doi.org/10.1007/978-3-319-11164-3_26
10. Karim, I., Al Ishtiaq, A., Hussain, S.R., Bertino, E.: Blediff: scalable and property-agnostic noncompliance checking for BLE implementations. *IEEE Symp. Secur. Priv.* **2023**, 3209–3227 (2023). <https://doi.org/10.1109/SP46215.2023.10179330>
11. Kim, S., Woo, S., Lee, H., Oh, H.: IoTCube: an automated analysis platform for finding security vulnerabilities. *IEEE Symp. Secur. Priv. Poster* (2017). https://wooseunghoon.github.io/assets/papers/SNP17_Poster.pdf
12. Lei, J., Wang, Y., Zhou, X., Yan, K.: BTFuzz: accurately fuzzing Bluetooth host with a simulated non-compliant controller. *IEEE Int. Conf. Civil Aviat. Safety Inf. Technol* **2022**, 1195–1201 (2022). <https://doi.org/10.1109/ICCASIT55263.2022.9987150>
13. Ma, R., Wang, D., Hu, C., Ji, W., Xue, J.: Test data generation for stateful network protocol fuzzing using a rule-based state machine. *Tsinghua Sci. Technol.* **21**(3), 352–360 (2016). <https://doi.org/10.1109/TST.2016.7488746>
14. Park, H., Nkuba, C.K., Woo, S., Lee, H.: L2Fuzz: discovering Bluetooth L2CAP vulnerabilities using stateful fuzz testing. *IEEE/IFIP Int. Conf. Dependable Syst. Netw* **2022**, 343–354 (2022). <https://doi.org/10.48550/arXiv.2208.00110>
15. Pferscher, A., Aichernig, B.K.: Stateful black-box fuzzing of Bluetooth devices using automata learning. *NASA Form. Methods Symp.* **2022**, 373–392 (2022). https://doi.org/10.1007/978-3-031-06773-0_20
16. Ruge, J., Classen, J., Gringoli, F., Hollick, M.: Frankenstein: advanced wireless fuzzing to exploit new Bluetooth escalation targets. *USENIX Secur. Symp.* 2020, 19–36 (2020). <https://www.usenix.org/conference/usenixsecurity20/presentation/ruge>
17. Seri, B., Vishnepolsky, G., Zusman, D.: BLEEDINGBIT: the hidden attack surface within BLE chips. Technical Report (2019). <https://i.blackhat.com/eu-18/Thu-Dec-6/eu-18-Seri-BleedingBit-wp.pdf>
18. Synopsys: Defensics fuzz testing. Available: <https://www.synopsys.com/software-integrity/security-testing/fuzz-testing.html>, last accessed March 25, 2025
19. Verde, D., Paiva, S., Lopes, S.: Assessing cybersecurity risks in BLE-based asset management systems. *IEEE Int. Conf. Syst. Signals Image Process* **2023**, 1–5 (2023). <https://doi.org/10.1109/IWSSIP58668.2023.10180264>
20. Wu, J., Nan, Y., Kumar, V., Tian, D.J., Bianchi, A., Payer, M., Xu, D.: BLESAs: spoofing attacks against reconnections in Bluetooth low energy. *USENIX Workshop Offens. Technol.* 2020 (2020). <https://www.usenix.org/conference/woot20/presentation/wu>
21. Wu, J., Wu, R., Xu, D., Tian, D.J., Bianchi, A.: Formal model-driven discovery of Bluetooth protocol design vulnerabilities. *IEEE Symp. Secur. Priv.* **2022**, 2285–2303 (2022). <https://doi.org/10.1109/SP46214.2022.9833777>
22. Zalewski, M.: AFL (American Fuzzy Lop). Available: <https://github.com/google/AFL>, last accessed March 25, (2025)
23. Zhao, B., Li, Z., Qin, S., Ma, Z., Yuan, M., Zhu, W., Zhang, C.: StateFuzz: system call-based state-aware Linux driver fuzzing. *USENIX Secur. Symp.* 2022, 3273–3289 (2022). <https://www.usenix.org/conference/usenixsecurity22/presentation/zhao-bodong>
24. Zou, Y.H., Bai, J.J., Zhou, J., Tan, J., Qin, C., Hu, S.M.: TCP-Fuzz: detecting memory and semantic bugs in TCP stacks with fuzzing. *USENIX Annu. Tech. Conf.* 2021, 489–502 (2021). <https://www.usenix.org/conference/atc21/presentation/zou>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.